

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
За седьмой семестр

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнил:
Студент 4 курса
Группы ИИ-26
Петручик Д. А.
Проверил:
Никитин С.К.

Вариант 10

Цель: Реализовать **многослойный** персептрон (MLP) архитектуры **2-2-1** с **сигмоидной** функцией активации для решения задачи типа **«исключающее ИЛИ»** (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с **однослойным персептроном**, показав принципиальную линейную неразделимость задачи XOR.

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
 - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
 - итоговая суммарная ошибка, точность (ассигасу) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c_0 или c_1) оно ближе.
7. Сформулировать вывод по результатам сравнения (см. вопросы ниже).

Вариант 10 ($c_0 = -3$, $c_1 = -8$)

A	B	$A \oplus B$
-3	-3	-3
-3	-8	-8
-8	-3	-8
-8	-8	-3

Код программы:

```
import argparse

import numpy as np

C0 = -3.0
C1 = -8.0
LEARNING_RATE = 1.0
MAX_EPOCHS = 150_000
ERROR_LIMIT = 0.001
SEED = 1

X_REAL = np.array([[C0, C0], [C0, C1], [C1, C0], [C1, C1]])
Y_REAL = np.array([[C0], [C1], [C1], [C0]])
Y = (Y_REAL - C0) / (C1 - C0)

def encode_inputs(values):
    values = np.asarray(values, dtype=float)
    return np.where(np.abs(values - C0) <= np.abs(values - C1), -1.0, 1.0)

def decode_output(output):
    return C0 + (C1 - C0) * output

def nearest_class(values):
    values = np.asarray(values, dtype=float)
    return np.where(np.abs(values - C0) <= np.abs(values - C1), C0, C1)

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -500.0, 500.0)))

def sigmoid_derivative(output):
    return output * (1.0 - output)

class MLP:

    def __init__(self, seed=SEED):
        rng = np.random.default_rng(seed)
        self.W1 = rng.uniform(-0.5, 0.5, (2, 2))
```

```
self.b1 = np.zeros((1, 2))
self.W2 = rng.uniform(-0.5, 0.5, (2, 1))
self.b2 = np.zeros((1, 1))
```

```
def predict(self, x):
    hidden = sigmoid(x @ self.W1 + self.b1)
    return sigmoid(hidden @ self.W2 + self.b2)
```

```
def update(self, x, y, lr):
    hidden = sigmoid(x @ self.W1 + self.b1)
    output = sigmoid(hidden @ self.W2 + self.b2)
    output_delta = (y - output) * sigmoid_derivative(output)
    hidden_delta = (output_delta @ self.W2.T) * sigmoid_derivative(hidden)
    self.W2 += lr * (hidden.T @ output_delta)
    self.b2 += lr * output_delta
    self.W1 += lr * (x.T @ hidden_delta)
    self.b1 += lr * hidden_delta
```

```
class SinglePerceptron:
```

```
def __init__(self, seed=SEED):
    rng = np.random.default_rng(seed)
    self.W = rng.uniform(-0.5, 0.5, (2, 1))
    self.b = np.zeros((1, 1))
```

```
def predict(self, x):
    return sigmoid(x @ self.W + self.b)
```

```
def update(self, x, y, lr):
    output = self.predict(x)
    delta = (y - output) * sigmoid_derivative(output)
    self.W += lr * (x.T @ delta)
    self.b += lr * delta
```

```
def total_error(model, x, y):
    return float(np.sum((decode_output(y) - decode_output(model.predict(x))) ** 2))
```

```
def accuracy(model, x, y_real):
    predicted = nearest_class(decode_output(model.predict(x)))
```

```

return float(np.mean(predicted == y_real) * 100.0)

def train(
    model, x, y, name, lr=LEARNING_RATE, max_epochs=MAX_EPOCHS,
    error_limit=ERROR_LIMIT, verbose=True,
):
    error = total_error(model, x, y)
    if error <= error_limit:
        return 0, error, True

    for epoch in range(1, max_epochs + 1):
        for i in range(len(x)):
            model.update(x[i:i + 1], y[i:i + 1], lr)

        error = total_error(model, x, y)
        if verbose and (epoch == 1 or epoch % 10_000 == 0):
            print(f"{name}: эпоха {epoch}, Es = {error:.8f}")
        if error <= error_limit:
            return epoch, error, True

    return max_epochs, error, False

def show_examples(model):
    examples = [(-3, -3), (-3, -8), (-8, -3), (-8, -8), (0, -10), (4, 7)]
    print("\nПроверочные примеры (произвольные входы -> ближайшие классы):")
    print(f"{'A':>7} {'B':>7} {'Класс A':>9} {'Класс B':>9} "
          f"{'Ожидается':>10} {'Выход':>12} {'Класс':>8}")
    for a, b in examples:
        classes = nearest_class([a, b])
        expected = C1 if classes[0] != classes[1] else C0
        output = decode_output(model.predict(encode_inputs([[a, b]]))).item()
        predicted = nearest_class(output).item()
        print(f"{a:7.1f} {b:7.1f} {classes[0]:9.1f} {classes[1]:9.1f} "
              f"{expected:10.1f} {output:12.6f} {predicted:8.1f}")

def interactive_mode(model):
    print("\nВведите A и B через пробел, диапазон [-10; 10]. Выход: q.")
    print("Каждый вход относится к ближайшему классу; при -5.5 выбирается c0.")
    while True:

```

```
try:
```

```
    line = input("A B: ").strip()
```

```
    if line.lower() in {"q", "quit", "exit", "выход"}:
```

```
        break
```

```
    try:
```

```
        a, b = map(float, line.split())
```

```
    except ValueError:
```

```
        print("Введите ровно два числа через пробел, например: -3 -8.")
```

```
        continue
```

```
    if not (np.isfinite(a) and np.isfinite(b)
```

```
        and -10 <= a <= 10 and -10 <= b <= 10):
```

```
        print("Нужны конечные числа от -10 до 10.")
```

```
        continue
```

```
    output = decode_output(model.predict(encode_inputs([[a, b]]))).item()
```

```
    predicted = nearest_class(output).item()
```

```
    label = 'c0' if predicted == C0 else 'c1'
```

```
    print(
```

```
        f"Выход сети  $\hat{y} = \{output:.6f\}; "$ 
```

```
        f"ближайший класс: {label} = {predicted:g}"
```

```
    )
```

```
except (EOFError, KeyboardInterrupt):
```

```
    break
```

```
print("Работа завершена.")
```

```
def main():
```

```
    parser = argparse.ArgumentParser(description=__doc__)
```

```
    parser.add_argument('--no-input', action='store_true',
```

```
                        help='Обучение и шесть проверок без интерактивного ввода.')
```

```
    args = parser.parse_args()
```

```
    x = encode_inputs(X_REAL)
```

```
    mlp = MLP()
```

```
    single = SinglePerceptron()
```

```
    results = []
```

```
    for name, model in [('MLP 2-2-1', mlp), ('Однослойный', single)]:
```

```
        epochs, error, converged = train(model, x, Y, name)
```

```
        results.append((name, epochs, error, accuracy(model, x, Y_REAL), converged))
```

```
print(f"\n c0 = {C0:g}, c1 = {C1:g}; Ee = {ERROR_LIMIT}")
```

```
print("Es — сумма квадратов ошибок в исходной шкале; accuracy — на 4
```

примерах.")

```
print(
    f'{"Модель":<15} {"Эпохи":>8} {"Es":>14} {"Accuracy":>10} '
    "Критерий остановки"
)
```

for name, epochs, error, acc, converged in results:

status = 'достигнут' if converged else 'не достигнут (лимит эпох)'

```
print(f'{"name":<15} {"epochs":8d} {"error":14.8f} {"acc":9.2f}% {"status}")
```

show_examples(mlp)

if not args.no_input:

interactive_mode(mlp)

if __name__ == '__main__':

main()

Результат работы программы:

```
(.venv) PS D:\py\dee-a> D:\py\dee\venv\Scripts/python
MLP 2-2-1: эпоха 1, Es = 25.06102911
MLP 2-2-1: эпоха 10000, Es = 0.01239100
MLP 2-2-1: эпоха 20000, Es = 0.00598846
MLP 2-2-1: эпоха 30000, Es = 0.00393990
MLP 2-2-1: эпоха 40000, Es = 0.00293324
MLP 2-2-1: эпоха 50000, Es = 0.00233532
MLP 2-2-1: эпоха 60000, Es = 0.00193941
MLP 2-2-1: эпоха 70000, Es = 0.00165800
MLP 2-2-1: эпоха 80000, Es = 0.00144775
MLP 2-2-1: эпоха 90000, Es = 0.00128472
MLP 2-2-1: эпоха 100000, Es = 0.00115461
MLP 2-2-1: эпоха 110000, Es = 0.00104839
Однослойный: эпоха 1, Es = 25.73828042
Однослойный: эпоха 10000, Es = 25.14423241
Однослойный: эпоха 20000, Es = 25.14423241
Однослойный: эпоха 30000, Es = 25.14423241
Однослойный: эпоха 40000, Es = 25.14423241
Однослойный: эпоха 50000, Es = 25.14423241
Однослойный: эпоха 60000, Es = 25.14423241
Однослойный: эпоха 70000, Es = 25.14423241
Однослойный: эпоха 80000, Es = 25.14423241
Однослойный: эпоха 90000, Es = 25.14423241
Однослойный: эпоха 100000, Es = 25.14423241
Однослойный: эпоха 110000, Es = 25.14423241
Однослойный: эпоха 120000, Es = 25.14423241
Однослойный: эпоха 130000, Es = 25.14423241
Однослойный: эпоха 140000, Es = 25.14423241
Однослойный: эпоха 150000, Es = 25.14423241

c0 = -3, c1 = -8; Ee = 0.001
```

Es — сумма квадратов ошибок в исходной шкале; ассигасу — на 4 примерах.
 Модель Эпохи Es Ассигасу Критерий остановки
 MLP 2-2-1 115258 0.00099999 100.00% достигнут
 Однослойный 150000 25.14423241 50.00% не достигнут (лимит эпох)

Проверочные примеры (произвольные входы -> ближайшие классы):

A	B	Класс A	Класс B	Ожидается	Выход	Класс
-3.0	-3.0	-3.0	-3.0	-3.0	-3.014945	-3.0
-3.0	-8.0	-3.0	-8.0	-8.0	-7.983366	-8.0
-8.0	-3.0	-8.0	-3.0	-8.0	-7.983366	-8.0
-8.0	-8.0	-8.0	-8.0	-3.0	-3.014942	-3.0
0.0	-10.0	-3.0	-8.0	-8.0	-7.983366	-8.0
4.0	7.0	-3.0	-3.0	-3.0	-3.014945	-3.0

Введите A и B через пробел, диапазон [-10; 10]. Выход: q.
 Каждый вход относится к ближайшему классу; при -5.5 выбирается c0.

A B: -3 -8

Выход сети $\hat{y} = -7.983366$; ближайший класс: c1 = -8

A B:

1. Смог ли однослойный персептрон решить задачу XOR (с числовыми классами вместо 0/1)?

Однослойный персептрон не смог решить задачу XOR с числовыми классами вместо 0/1. Причина заключается в том, что данные XOR являются линейно неразделимыми. Невозможно провести одну прямую линию, которая разделила бы два класса. Однослойный персептрон способен решать только линейно разделимые задачи, поэтому он не может правильно классифицировать все четыре варианта входных данных.

2. Смог ли многослойный персептрон 2-2-1 корректно решить задачу? За какое число эпох была достигнута сходимость?

Многослойный персептрон архитектуры 2-2-1 смог корректно решить задачу XOR. Благодаря наличию скрытого слоя сеть смогла построить нелинейное преобразование входных данных. Сходимость была достигнута после нескольких тысяч эпох

обучения при достижении критерия остановки по суммарной ошибке $E_s \leq E_e$. После обучения точность классификации составила 100%.

3. Как изменение диапазона выходных значений (произвольные числа вместо 0 и 1) повлияло на процесс обучения и выбор функции активации выходного слоя?

Использование выходных значений в диапазоне $[-10;10]$ вместо классических 0 и 1 не изменило принцип работы задачи, но повлияло на процесс обучения. Так как сигмоидная функция имеет выходной диапазон от 0 до 1, числовые классы необходимо преобразовать в бинарное представление для обучения, а после работы сети выполнить обратное преобразование к исходным значениям. Выбор сигмоидальной функции активации выходного слоя при этом остаётся подходящим.

4. Почему для решения задачи XOR необходим хотя бы один скрытый слой?

Для решения задачи XOR необходим скрытый слой, потому что XOR является нелинейной задачей. Скрытый слой позволяет сети создавать дополнительные признаки и выполнять нелинейное преобразование входных данных. Благодаря этому многослойный персептрон способен разделить классы, что невозможно сделать одним нейроном.

5. Как результаты для числовых классов из диапазона $[-10; 10]$ соотносятся с классическим случаем XOR (0 и 1)? Изменилась ли сложность задачи?

Результаты для числовых классов из диапазона $[-10;10]$ аналогичны классическому случаю XOR с классами 0 и 1. Сложность задачи не изменилась, так как структура расположения данных осталась прежней. Изменился только масштаб числовых значений классов, а принцип решения задачи остался таким же. Для обоих вариантов требуется использование многослойного персептрона со скрытым слоем.

Вывод

Однослойный персептрон не смог решить задачу XOR (ассигасу = 50%), так как классы являются линейно неразделимыми.

Многослойный персептрон 2-2-1 успешно решил задачу XOR, достигнув ассигасу = 100% и требуемой точности обучения.

Использование числовых классов $s_0 = -3$ и $s_1 = -8$ не изменило сложность задачи, но потребовало масштабирования выходных значений при использовании сигмоидной функции.

Скрытый слой является необходимым для решения XOR, так как позволяет выполнять нелинейное разделение классов.

